

4. Evaluación y testing de Interfaces Persona Ordenador

Verificación y validación (cód. 30244)

Miguel Ángel Latre
Javier Nogueras Iso

Departamento de Informática e Ingeniería de Sistemas
Universidad de Zaragoza



1. Introducción
2. Automatización de pruebas con GUI
3. Automatización de pruebas de aceptación de usuarios
4. Soporte para pruebas de usabilidad
5. Soporte para pruebas de accesibilidad



1. Introducción

2. Automatización de pruebas con GUI

3. Automatización de pruebas de aceptación de usuarios

4. Soporte para pruebas de usabilidad

5. Soporte para pruebas de accesibilidad



1. Introducción

- Todos los sistemas utilizados por humanos tienen algún tipo de interfaz de usuario (IU, también Interfaz Persona Ordenador), gráfica o no gráfica
- Tipos de pruebas donde la Interfaz Persona Ordenador resulta especialmente relevante:
 - Pruebas funcionales (o regresión)
 - Pruebas dirigidas a evaluar si el sistema es conforme a los requisitos funcionales
 - En algunos sistemas poco modularizados las pruebas se realizan directamente sobre la IU ya que no es posible interactuar con el API de módulos o componentes separados del sistema
 - A nivel de pruebas de sistema y aceptación la interacción directa con la interfaz de usuario es la forma de interacción más habitual

• ...

- Pruebas de usabilidad

- Usabilidad: medida en la que un producto se puede utilizar por determinados usuarios para conseguir objetivos específicos con efectividad, eficiencia y satisfacción en un contexto de uso especificado (ISO 3241-11)
- La IU es la puerta del usuario a la funcionalidad del sistema subyacente (la IU es clave a la hora de evaluar la usabilidad)
- Una IU mal diseñada es un factor que frena el uso de las funcionalidades
- Es muy importante diseñar IU usables

- Pruebas de accesibilidad

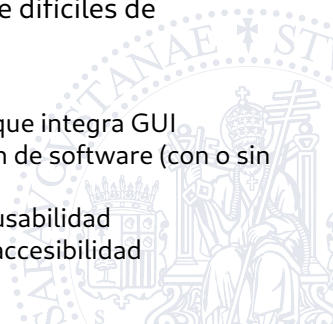
- Accesibilidad: grado en el que un producto, dispositivo, servicio o entorno está disponible para tantas personas como sea posible
- La IU es un factor clave en la accesibilidad de un producto

- Pruebas de portabilidad

- La portabilidad hace referencia a la capacidad de un sistema para ejecutarse sobre un conjunto amplio de configuraciones hardware y software
- La IU es especialmente sensible a los cambios de plataforma de ejecución

Objetivos de este tema

- En la asignatura de Interacción Persona-Ordenador ya se han visto conceptos y técnicas de evaluación
- En esta asignatura se presentan herramientas que permitan la ejecución automática de pruebas para sistemas con interfaces gráficas de usuario
- Las interfaces gráficas son particularmente difíciles de automatizar
- Aspectos concretos a tratar
 - Automatización de pruebas de software que integra GUI
 - Automatización de pruebas de aceptación de software (con o sin GUI)
 - Soporte para la ejecución de pruebas de usabilidad
 - Soporte para la ejecución de pruebas de accesibilidad



Índice

1. Introducción
2. Automatización de pruebas con GUI
3. Automatización de pruebas de aceptación de usuarios
4. Soporte para pruebas de usabilidad
5. Soporte para pruebas de accesibilidad



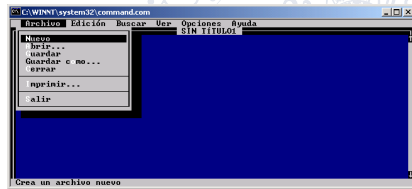
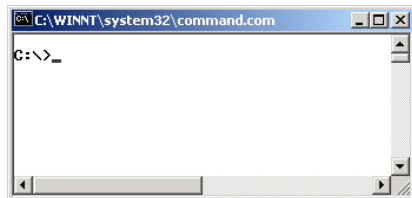
2. Automatización de pruebas con GUI

- Testabilidad (*testability*)
 - la facilidad para tener interfaces adecuadas y fiables para dirigir la ejecución de las pruebas y su evaluación
 - la facilidad y la rapidez con las cuales se pueden probar las características de un sistema
- Para una testabilidad alta se requiere control (poder ejecutar partes bien definidas del sistema) y visibilidad (verificar el resultado)
 - La clave para la testabilidad es la colaboración estrecha entre profesionales de las pruebas y los desarrolladores
 - Los subsistemas deberían seguir un patrón *Facade*
- La testabilidad de cualquier interfaz de programación (API) es alta porque las herramientas de pruebas pueden interactuar con ellas de forma similar a otros programas

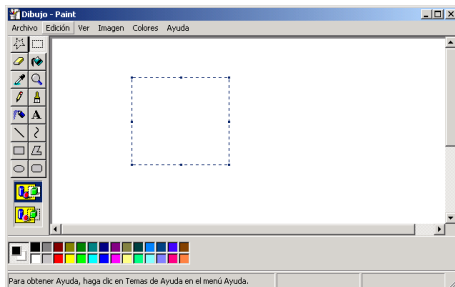
Testabilidad de las Interfaces Persona Ordenador

- Interfaces no gráficas

- Interfaces por línea de comandos (ej : Sistemas Operativos como MS-Dos , o Unix)
 - Alta testabilidad
 - Ejemplos de herramientas de automatización: utilización de shell scripts; redirigir entrada y salida estándar en herramientas como JUnit (ya se vio en práctica 3)
- Interfaces textuales (ej : Edit de MSDos)
 - No son triviales pero se pueden utilizar herramientas simples para controlarlas
 - Ejemplos de herramientas de automatización: Expect

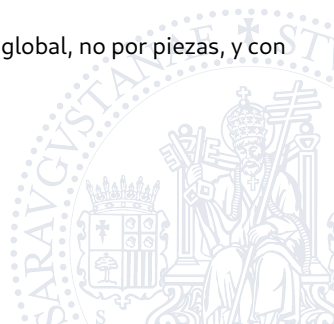


- Interfaces gráficas (Interfaces gráficas de usuario o GUI)
 - Por norma general no están diseñadas para la automatización y son técnicamente complicadas
 - Si es posible, los expertos recomiendan minimizar la automatización de las pruebas de GUI
 - Se recomienda separar la arquitectura del sistema en capas de presentación y de dominio (negocio)
 - La capa de dominio se puede automatizar fácilmente
 - La capa de presentación (cuando es muy simple) se podría revisar manualmente con poco esfuerzo y observar también aspectos relacionados con la usabilidad



- Ejemplos de variantes del patrón Model View Controller (MVC) que facilitan la testabilidad (creación de tests automáticos)
 - Modelo-Vista-Presentador (Vista Pasiva)
 - <https://martinfowler.com/eaDev/PassiveScreen.html>
 - “El presentador (controlador) responde a los eventos del usuario y es responsable de actualizar la vista. Todo el comportamiento específico de la aplicación se extrae al controlador.”
 - Modelo-Vista-Presentador (Presentador Supervisor)
 - <https://martinfowler.com/eaDev/SupervisingPresenter.html>
 - “El Presentador Supervisor (controlador) responde a los eventos del usuario y lleva a cabo parte de la sincronización entre modelo y vista. Solo las correspondencias sencillas entre modelo y vista se gestionan directamente desde la vista.”
 - Presentados en asignatura de Arquitecturas Software
 - En ambos patrones la lógica de la interacción (controlador) se separa de la representación gráfica (vista)

- Pero hay ocasiones donde es necesaria la automatización de las pruebas de GUI
 - Interfaces muy complejas
 - Sistemas monolíticos poco modularizados
 - Sistemas legados cuyo diseño se desconoce
 - Pruebas de aceptación de usuarios
 - nos interesa probar el sistema de forma global, no por piezas, y con una batería amplia de casos de prueba

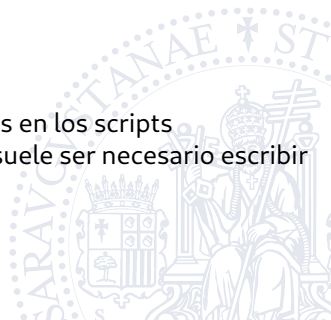


Generaciones de entornos de automatización

- Automatización de las pruebas (*test automation*)
 - El uso de software para controlar la ejecución de las pruebas, la comparación de los resultados reales respecto a los esperados, la configuración de las precondiciones de las pruebas y otras funcionalidades relacionadas con el control de las pruebas y la elaboración de informes
- La escala de automatización de las pruebas puede variar
 - desde la utilización de programas aislados para facilitar pequeñas partes de las pruebas (ej: *diff* para comparar ficheros, *grep*, ...)
 - a sistemas que facilitan prácticamente todo: configuración del entorno, ejecución de pruebas, informes
- Se suelen distinguir 3 generaciones en los entornos de automatización de pruebas con GUI
 - [Kit E (1999). *Integrated, effective test design and automation*. *Software Development*, 21(2):36–38.]

Primera generación

- Los scripts se suelen generar con herramientas *capture and replay*, pero se pueden codificar también manualmente
 - Herramientas *Capture and replay* (o *record and playback*): capturan las acciones en una ejecución manual de una aplicación, y las transcriben a un script para repetir después de forma automática esa ejecución
- Desventajas
 - Herramientas poco estructuradas
 - Los datos de las pruebas están embebidos en los scripts
 - Cuando se modifica el sistema a probar, suele ser necesario escribir (capturar) de nuevo los scripts



Segunda generación

- Los scripts están bien diseñados, se fomenta modularidad, son robustos, están bien documentados
 - Los scripts son mantenibles
 - Se suelen definir librerías de tareas (*task libraries*) que agrupan secuencias de pasos que aparecen en varios casos de pruebas:

```
system_login("juan23","secreta"):  
  menu_select("System login")  
  enter_text("Login_Name", "juan23")  
  enter_text("Password", "secreta")  
  push_button("Ok")
```

- Los scripts no solo gestionan la ejecución, sino que también facilitan la configuración inicial, la detección de errores y recuperación, y las acciones de finalización

- Desventajas

- Los datos de las pruebas están todavía embebidos en los scripts
 - Cada caso de pruebas requiere un script
- El código se escribe manualmente, y la implementación y el mantenimiento requieren habilidades de programación que puede que los expertos de las pruebas no tengan



Tercera generación

- Mantienen las características buenas de la 2ª generación
- Además, los datos de las pruebas están fuera de los scripts
 - Un mismo script puede ejecutar casos de prueba similares
 - El diseño de las pruebas y su implementación son dos tareas separadas que se pueden asignar a recursos diferentes
 - Un experto en el dominio puede realizar el diseño de la pruebas
 - Un programador se puede encargar de codificar el script
- Esta forma de programar los scripts se conoce como *data-driven testing* o *data-driven test automation* (un concepto similar a los tests parametrizados de Junit)

username	password
maveryx	maveryx
maveryx123	maveryx123
maveryx@maveryx.com	maveryx@maveryx.com
mvxo1	mvxo
maveryx@maveryx.org	maveryx@maveryx.it

- También suelen facilitar la generación automática de scripts en base a palabras clave (*keyword-driven testing*)
 - Generar scripts con palabras clave que representan directivas diciendo qué es lo que hay que hacer y los parámetros necesarios
 - Desventaja: se necesita un parser o un script capaz de interpretar las palabras clave

Test 1		
Login	juan23	secreta
VerifyScreen	MainAccount	
Logoff		

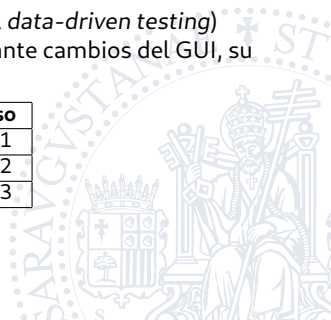


- Otras características deseables para herramientas de prueba de GUI

- *Window mapping*

- Se trata de utilizar nombres específicos para los componentes del GUI evitando el método de acceso concreto (etiquetas adyacentes, orden en la ventana, posición absoluta, otras propiedades de los componentes, identificadores internos, ...)
- Las correspondencias se establecen en tablas o ficheros de propiedades separados (algo parecido al *data-driven testing*)
- Facilita la mantenibilidad de los scripts ante cambios del GUI, su internacionalización, ...

Nombre	Método de acceso
Login_Name	Caja de texto nº 1
Server	Caja de texto nº 2
Password	Caja de texto nº 3



- *Object-aware GUI testing tools*

- Capaces de detectar e identificar componentes del GUI conforme se ejecutan las aplicaciones
- Ejemplo: pueden detectar que un botón de Ok está presente antes de pulsarlo
- Esta característica no estaba disponible en primeras herramientas y había problemas de sincronización

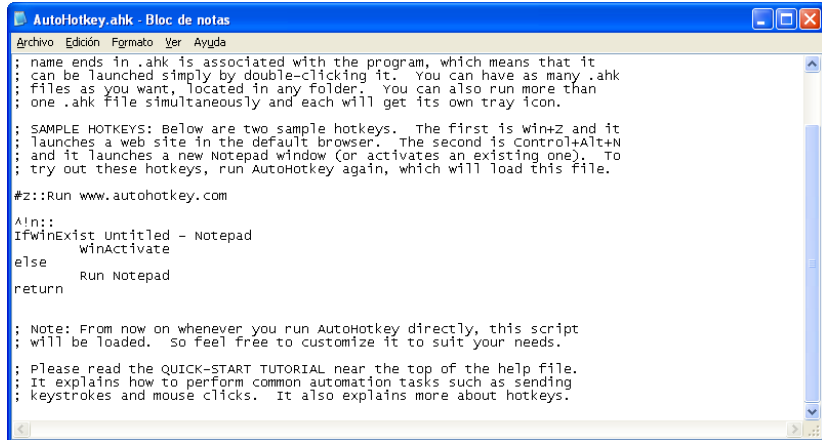


Algunas herramientas Open Source

- https://en.wikipedia.org/wiki/Comparison_of_GUI_testing_tools
- Primera generación
 - AutoHotkey (Windows), AutoKey (Linux): teclas rápidas que lanzan scripts para automatizar la interacción con la GUI (envío de teclas, clics, ...)
 - Exerciser Monkey: pruebas de estrés para aplicaciones Android
 - SikuliX: Prueba de cualquier tipo de aplicación o página Web en base a capturas de pantalla
- Segunda generación
 - Selenium : Prueba de aplicaciones Web
- Tercera generación
 - Espresso , UI Automator : Prueba de aplicaciones Android



- <https://www.autohotkey.com/>



```
AutoHotkey.ahk - Bloc de notas
Archivo Edición Formato Ver Ayuda

; name ends in .ahk is associated with the program, which means that it
; can be launched simply by double-clicking it. You can have as many .ahk
; files as you want, located in any folder. You can also run more than
; one .ahk file simultaneously and each will get its own tray icon.

; SAMPLE HOTKEYS: Below are two sample hotkeys. The first is Win+Z and it
; launches a web site in the default browser. The second is Control+Alt+N
; and it launches a new Notepad window (or activates an existing one). To
; try out these hotkeys, run AutoHotkey again, which will load this file.

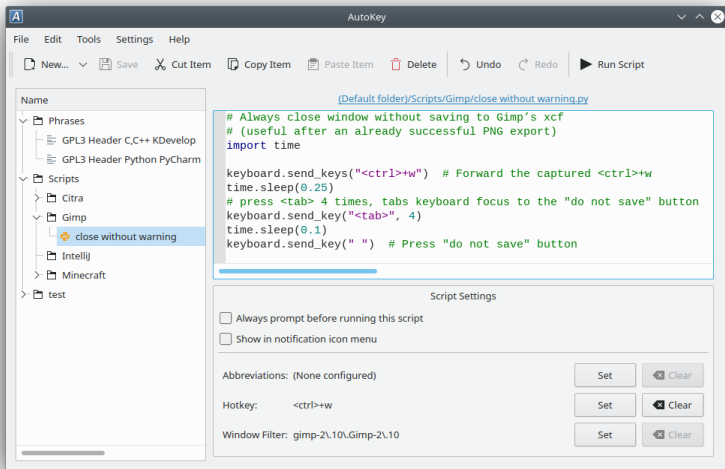
#z::Run www.autohotkey.com

^!n::
IfWinExist Untitled - Notepad
    WinActivate
else
    Run Notepad
return

; Note: From now on whenever you run AutoHotkey directly, this script
; will be loaded. So feel free to customize it to suit your needs.

; Please read the QUICK-START TUTORIAL near the top of the help file.
; It explains how to perform common automation tasks such as sending
; keystrokes and mouse clicks. It also explains more about hotkeys.
```

- <https://github.com/autokey/autokey>

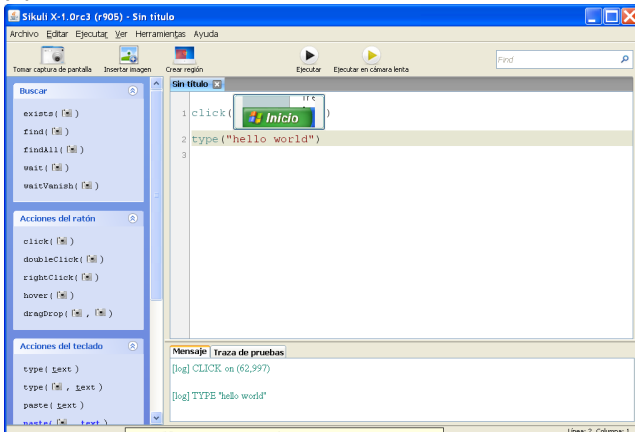


Exerciser Monkey

- Herramienta proporcionada por Android para realizar pruebas de estrés
- Envía secuencias de eventos pseudoaleatorios (clics, toques, gestos y eventos de sistemas) a la interfaz de la aplicación que se desea probar, tanto en el emulador como en un dispositivo real
- `adb/platform-tools/adb`

```
adb shell monkey -p paqueteDeLaApp -v numeroDeEventos
```

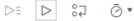

- <http://sikulix.com/>
- Facilita la prueba de cualquier tipo de aplicación o página Web en base a capturas de pantalla
- Es multiplataforma
- No hay posibilidad de realizar aserciones



- <https://www.selenium.dev/>
- Facilita un conjunto de herramientas para automatizar de pruebas de GUI con aplicaciones Web
- Selenium IDE es un plug -in de Firefox y Chrome
 - Es un entorno que no requiere conocimientos de un lenguaje de programación específico
 - Incluye una herramienta de tipo *Capture and Replay* para generar los scripts de forma automática
 - Se facilitan comandos de tipo *Verify* (continúa la ejecución en caso de fallo) y *Assert* (aborta la ejecución en caso de fallo)
 - Varias posibilidades para localización de elementos de una página Web
- Selenium WebDriver es un driver con distintas implementaciones para interactuar con diferentes navegadores
 - Posteriormente se puede escribir scripts en varios lenguajes (Java, C#, Ruby, Python) para manejar la conexión (WebDriver) con el navegador

moz-extension://2483a6b1-ef5e-44cc-9fcb-c81f2927b93b - Selenium IDE - Untitled Project - Mozilla Firefox

Untitled Project

Tests + 

Search tests... 

Untitled	Command	Target	Value
5.	click at	css=h2.nomargin > a	79,28
6.	click at	id=qNoticia	112,14
7.	type	id=qNoticia	elecciones
8.	click at	//input[@value='CONSULTAR']	64,13
9.	click at	//a[contains(text(),'Limpiar filtros')]	19,11

Command:

Target:  

Value:

Comment:

Runs: 1 Failures: 0

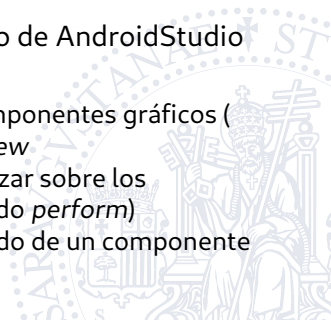
Log

Running 'Untitled'

- Trying to execute open on /sedeelectronica/... **Success**
- Trying to execute mouseDownAt on css=h2.nomargin > a with value 79,28.083328247070312... **Success**
- Trying to execute mouseMoveAt on css=h2.nomargin > a with value 79,28.083328247070312... **Success**
- Trying to execute mouseUpAt on css=h2.nomargin > a with value 79,28.083328247070312... **Success**
- Trying to execute clickAt on css=h2.nomargin > a with value 79,28... **Success**
- Trying to execute clickAt on id=qNoticia with value 112,14... **Success**
- Trying to execute type on id=qNoticia with value elecciones... **Success**
- Trying to execute clickAt on //input[@value='CONSULTAR'] with value 64,13... **Success**
- Trying to execute clickAt on //a[contains(text(),'Limpiar filtros')] with value 19,11... **Success**

'Untitled' completed successfully

- <https://developer.android.com/training/testing/espresso/>
- Facilita la automatización de pruebas de GUI para aplicaciones Android
- Se dice que los script de pruebas son de tipo caja blanca porque conocen y utilizan detalles del código de la aplicación que se está probando
- Pruebas instrumentadas integradas dentro de AndroidStudio
- Elementos principales
 - *ViewMatchers*: objetos para localizar componentes gráficos (*ViewInteraction*) gracias al método *onView*
 - *ViewActions*: acciones que se pueden lanzar sobre los componentes gráficos (a través del método *perform*)
 - *ViewAssertions*: aserciones sobre el estado de un componente gráfico (a través del método *check*)



Ejemplo de test con Espresso

● <https://www.vogella.com/tutorials/AndroidTestingEspresso/article.html>

@RunWith (AndroidJUnit4.class)

```
public class MainActivityEspressoTest {
```

```
    @Rule
```

```
    public ActivityScenarioRule<MainActivity> mActivityRule = new  
        ActivityScenarioRule<>(MainActivity.class); //Se lanza la  
        actividad antes del inicio de los tests y se cierra despues
```

```
    @Test
```

```
    public void ensureTextChangesWork() {
```

```
        // Type text and then press the button.
```

```
        onView(withId(R.id.inputField)).perform(typeText("hello"),  
            closeSoftKeyboard());
```

```
        onView(withId(R.id.changeText)).perform(click());
```

```
        // Check that the text was changed.
```

```
        onView(withId(R.id.inputField)).check(matches(withText("Lalala")  
            ));
```

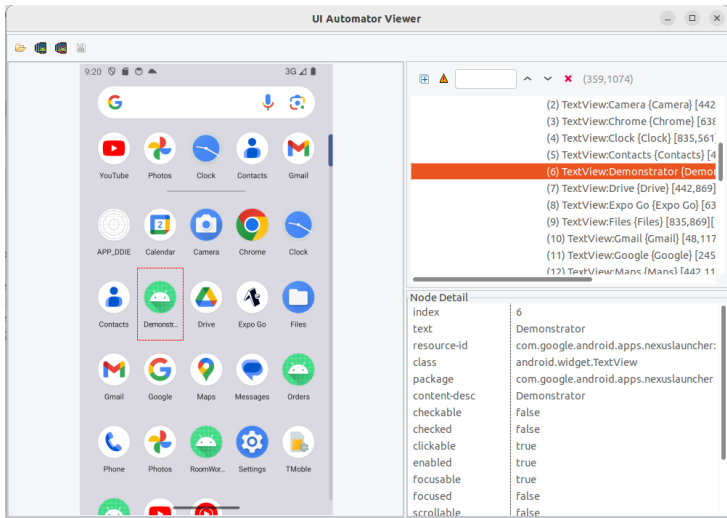
```
    }
```

```
}
```



- <https://developer.android.com/training/testing/other-components/ui-automator>
- Facilita la automatización de pruebas de GUI para aplicaciones instaladas en dispositivos Android
- Se dice que los script de pruebas son de tipo caja negra porque no utilizan los detalles internos de la aplicación a probar
 - Este *framework* está pensado para probar aplicaciones que enlazan con utilidades del sistema u otras aplicaciones instaladas en el dispositivo
- Elementos principales
 - *UI Automator Viewer*: utilidad que permite descubrir la jerarquía de elementos gráficos visibles en una pantalla
 - *UiDevice*: objeto para acceder al dispositivo, conocer su estado y enviar acciones
 - APIs para seleccionar y acceder a elementos gráficos (*By*, *UiObject2*, ...)

Ejemplo de test con UI Automator



```
private static final int TIMEOUT = 5000;
private static String APP_NAME = "Demonstrator";
...
// Se inicializa la instancia de UiDevice
mDevice = UiDevice.getInstance(InstrumentationRegistry.
    getInstrumentation());
// Se pulsa el boton para ir a la pantalla inicial
mDevice.pressHome();
// Nos deslizamos por la pantalla en sentido vertical (por defecto) u
    horizontal
UiScrollable appViews = new UiScrollable(new UiSelector().scrollable(
    true));
appViews.scrollForward();
// Abrir la aplicacion
mDevice.wait(Until.hasObject(By.desc(APP_NAME)), TIMEOUT);
UiObject2 app = mDevice.findObject(By.desc(APP_NAME));
assertThat(app, notNullValue());
app.clickAndWait(Until.newWindow(), TIMEOUT);
```

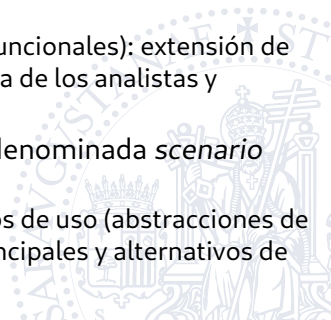

Índice

1. Introducción
2. Automatización de pruebas con GUI
3. Automatización de pruebas de aceptación de usuarios
4. Soporte para pruebas de usabilidad
5. Soporte para pruebas de accesibilidad

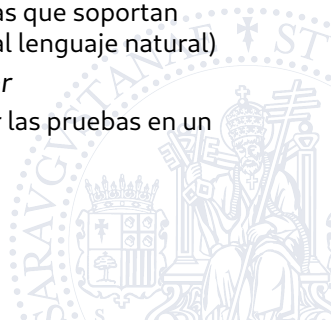


3. Automatización de pruebas de aceptación de usuarios

- En las pruebas de aceptación se intenta facilitar la colaboración del cliente/usuario
- Suelen consistir en el diseño y ejecución de casos de prueba inspirados en los requisitos de usuario
 - Requisitos de usuario: requisitos expresados en lenguaje natural a alto nivel
 - Requisitos de sistema (funcionales y no funcionales): extensión de los anteriores pero desde el punto de vista de los analistas y elaborados con mayor nivel de detalle
- Se aplica la técnica de diseño de pruebas denominada *scenario testing* (ISO 29119)
 - Tomando como punto de partida los casos de uso (abstracciones de escenarios), se transforman los flujos principales y alternativos de cada caso de uso en casos de prueba



- Pero para facilitar la colaboración (control) del usuario en estas pruebas nos interesaría que el usuario transformase de forma automática los casos de prueba en scripts ejecutables
- Las herramientas mostradas hasta el momento requieren ciertas habilidades de programación para definición de los scripts que automatizan la ejecución de los casos de prueba
 - La única excepción serían las herramientas que soportan *keyword-driven testing* (mayor cercanía al lenguaje natural)
- Propuesta de un nuevo entorno: *Cucumber*
 - Facilitar que el usuario final pueda definir las pruebas en un lenguaje casi natural



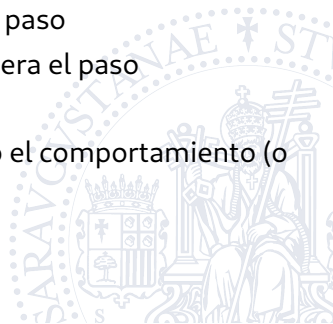
Cucumber

- <https://cucumber.io/>
- Permite ejecutar descripciones funcionales en texto plano como pruebas automatizadas
- En realidad, en lugar de hablar de una herramienta de pruebas podríamos hablar de *Behaviour-Driven Development (BDD)*
 - Desarrollar en base al comportamiento esperado de una aplicación
- La herramienta Cucumber está desarrollada con el lenguaje Ruby pero permite probar código desarrollado en multitud de lenguajes (Java, C, Python , .NET, ...)



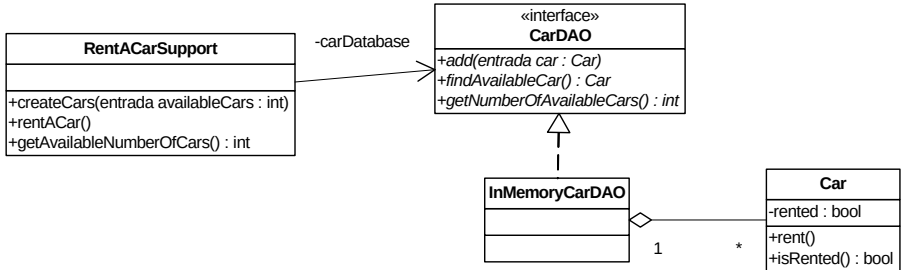
Metodología de trabajo de BDD con Cucumber

- 1 Describir el comportamiento esperado en texto plano utilizando un lenguaje mínimamente controlado (Gherkin)
- 2 Escribir la definición de un paso
- 3 Ejecutar y observar si el test falla
- 4 Escribir el código que permita superar ese paso
- 5 Ejecutar de nuevo y comprobar que se supera el paso
- 6 Repetir 2-5 hasta que se supera el test
- 7 Repetir 1-6 hasta que se implementa todo el comportamiento (o se acaba el presupuesto)



Ejemplo: una aplicación Java para alquiler de coches

- O aplico BDD para desarrollar, o defino pruebas de aceptación para un sistema ya desarrollado
- Requisitos funcionales del sistema:
 - Buscar coches para alquilar
 - **Alquilar un coche**
 - Devolver un coche
 - Cargar el importe del alquiler de un coche
 - Mantener un inventario de coches



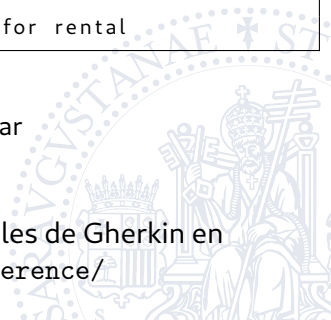
● "1. Descripción del comportamiento esperado"

Feature: Rental cars should be possible to rent to gain revenue to the rental company.

As an owner of a car rental company
I want to make cars available for renting
So I can make money

Scenario: Find and rent a car
Given there are 18 cars available for rental
When I rent one
Then there will only be 17 cars available for rental

- Tres componentes en el escenario
 - Given: precondiciones del sistema a probar
 - When: el cambio real sobre el sistema
 - Then: el estado esperado del sistema
- Consultar la sintaxis de keywords adicionales de Gherkin en <https://cucumber.io/docs/gherkin/reference/>



● “2. Escribir la definición de un paso”

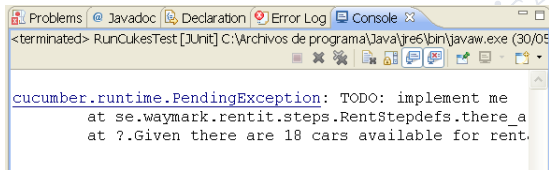
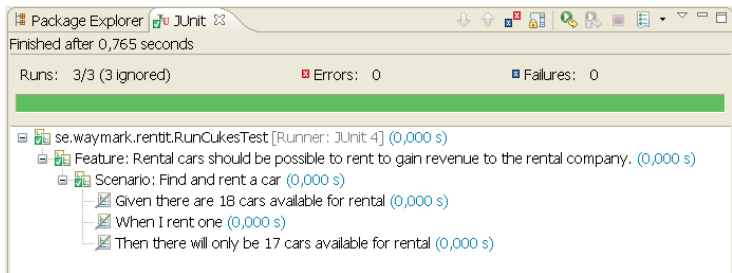
- Cucumber-JVM permite ejecutar pruebas sobre aplicaciones Java
- Incluye un parser que traduce el lenguaje Gherkin en el esqueleto de los métodos que permiten la definición de un paso
- Se puede ejecutar el parser con JUnit

```
@Given("^there_are_(\\d+)_cars_available_for_rental$")
public void there_are_cars_available_for_rental(int arg1) throws
    Throwable {
    // Express the Regexp above with the code you wish you had
    throw new PendingException();
}

@When("^I_rent_one$")
public void I_rent_one() throws Throwable {
    // Express the Regexp above with the code you wish you had
    throw new PendingException();
}

@Then("^there_will_only_be_(\\d+)_cars_available_for_rental$")
public void there_will_only_be_cars_available_for_rental(int arg1)
    throws Throwable {
    // Express the Regexp above with the code you wish you had
    throw new PendingException();
}
```


● “3. Ejecutar y observar si el test falla”



- "4. Escribir el código que permita superar ese paso"

```
public class RentStepdefs {  
    private RentACarSupport rentACarSupport = new RentACarSupport();  
  
    @Given("^there_are_(\\d+)_cars_available_for_rental$")  
    public void there_are_cars_available_for_rental(int availableCars)  
        throws Throwable {  
        rentACarSupport.createCars(availableCars);  
    }  
  
    @When("^I_rent_one$")  
    public void rent_one_car() throws Throwable {  
        rentACarSupport.rentACar();  
    }  
  
    @Then("^there_will_only_be_(\\d+)_cars_available_for_rental$")  
    public void there_will_be_less_cars_available_for_rental(int  
        expectedAvailableCars) throws Throwable {  
        int actualAvailableCars = 16; // error a posta  
        assertThat(actualAvailableCars, is(expectedAvailableCars));  
    }  
}
```

- “5. Ejecutar de nuevo y comprobar que se supera el paso”

Package Explorer JUnit x

Finished after 0,25 seconds

Runs: 3/3 Errors: 0 Failures: 2

se.waymark.rentit.RunCukesTest [Runner: JUnit 4] (0,047 s)

- Feature: Rental cars should be possible to rent to gain revenue to the rental company. (0,047 s)
 - Scenario: Find and rent a car (0,047 s)
 - Given there are 18 cars available for rental (0,000 s)
 - When I rent one (0,016 s)
 - Then there will only be 17 cars available for rental (0,031 s)

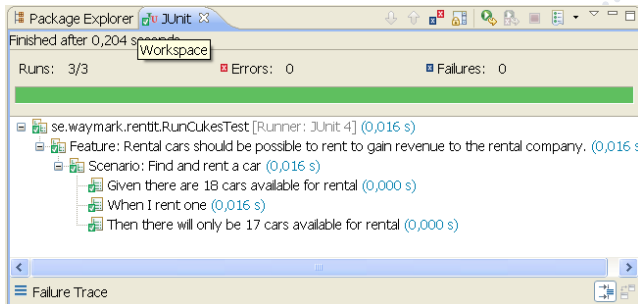
Failure Trace

java.lang.AssertionError:
Expected: is <17>
got: <16>

at se.waymark.rentit.steps.RentStepdefs.there_will_be_less_cars_available_for_rental(RentStepdefs.java:10)
at □.Then there will only be 17 cars available for rental(se\waymark\rentit\Rent.feature:10)

- “Corrigiendo posibles problemas (repetir 2-5)”

```
public class RentStepdefs {  
    @Then("there will only be (\\d+) cars available for rental$")  
    public void there_will_be_less_cars_available_for_rental(int  
        expectedAvailableCars) throws Throwable  
  
    {  
        int actualAvailableCars = rentACarSupport.  
            getAvailableNumberOfCars();  
        assertThat(actualAvailableCars, is(expectedAvailableCars));  
    }  
}
```



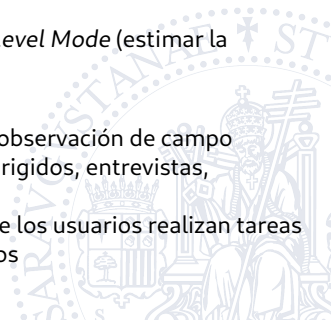
Índice

1. Introducción
2. Automatización de pruebas con GUI
3. Automatización de pruebas de aceptación de usuarios
4. Soporte para pruebas de usabilidad
5. Soporte para pruebas de accesibilidad



4. Soporte para pruebas de usabilidad

- Sistema usable: fácil de aprender y fácil de utilizar
- En la asignatura de IPO se han presentado distintos métodos de evaluación
 - Sin usuarios (inspección)
 - Heurísticas: guías y estándares
 - Recorridos cognitivos: recorrido de acciones necesarias para ejecutar una tarea
 - Modelos predictivos: KLM – *Keystroke Level Mode* (estimar la eficiencia de un sistema), ...
 - Con usuarios (indagación y tests)
 - Observacionales: pensando en voz alta, observación de campo
 - De interrogación: grupos de discusión dirigidos, entrevistas, cuestionarios
 - Experimentales: test de usabilidad donde los usuarios realizan tareas en entornos controlados y se toman datos



- A priori, no hay mucha posibilidad de automatización, salvo:
 - Evaluación heurística (ver después herramientas de evaluación de accesibilidad Web)
 - Toma de datos para métodos con usuarios
 - Captura de datos (logs) y análisis
 - Facilitar gestión de cuestionarios
 - Ej: SurveyMonkey, Google Forms



2. How would you describe your expertise with computers and the web?

☐ Beginner: Others help me with website problems more often than I help them.

☐ Intermediate: I help others with website problems more often than they help me.

☐ Advanced: I have so much experience with the web that I feel comfortable giving advice to people who build websites.

3. On a typical day, how much time do you spend using the web?

☐ Less than 1 hour

☐ 1-3 hours

☐ 3-5 hours

☐ 5-7 hours

☐ More than 7 hours

Herramientas de analítica web

- Objetivos de estas herramientas:
 - Captación de datos sobre el uso de una web.
 - Exposición de los datos obtenidos
- No solo se trata de tomar medidas:
 - Número de visitas o visitantes
 - Cantidad de URL/Páginas vistas
 - Tiempo invertido en el sitio
- Se trata de:
 - Cuantificar el cumplimiento de objetivos
 - Medir la fidelidad de los usuarios
 - Identificar los grupos de usuarios y sus tendencias de uso del servicio
 - Segmentar la información obtenida

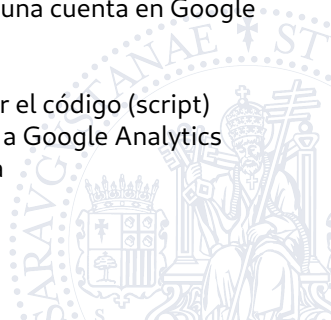


- Algunos ejemplos

- Estadísticas ad-hoc en base a logs de servidores Web (ej: Apache, ...)
- Software a instalar en servidor: AWStats
- Servicios proporcionados por organizaciones externas: Google Analytics , Yahoo Web Analytics , StatCounter.com

- ¿Cómo funciona Google Analytics ?

- El propietario del sitio Web debe crearse una cuenta en Google Analytics
- <https://analytics.google.com>
- El desarrollador del sitio Web debe añadir el código (script) necesario en el sitio para enviar los datos a Google Analytics asociados a la cuenta previamente creada
- ...



Google Analytics - Mozilla Firefox

Archivo Editar Ver Historial Marcadores Herramientas Ayuda

Get started with Analytics - Anal... Google Analytics

https://www.google.com/analytics/web/provision?et=8authuser=#provision/CreateAccount/ Web Search

Setting up your web property

Website Name

Cliente OGC avanzado

Web Site URL

http:// http://tiernas.cps.unizar.es:8088/csw/client.html

Example: http://www.mywebsite.com

Industry Category new ?

We've added more Industry Categories! Select one that best represents your business.

Other

Reporting Time Zone

Spain (GMT+01:00) Madrid

Setting up your account

Account Name

Accounts are the top-most level of organization and contain one or more tracking IDs.

verificacionValidacion

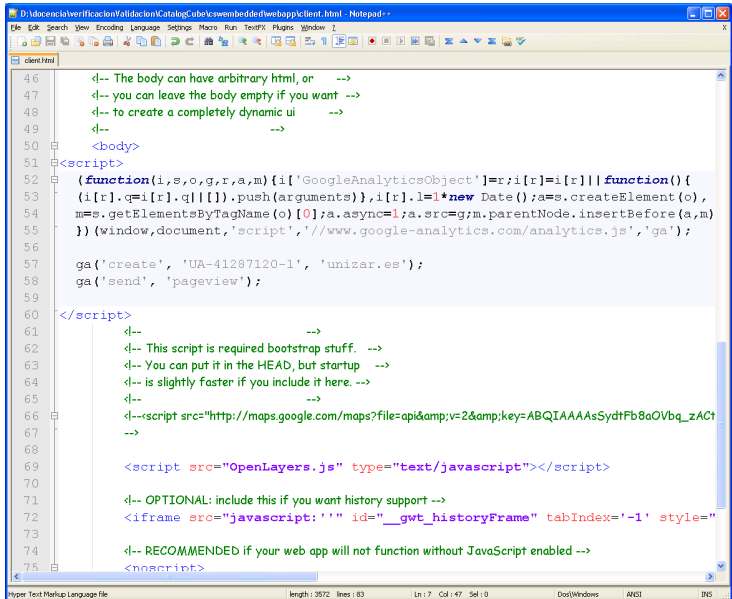
Data Sharing Settings ?

- ☒ **With other Google products only** optional
Enable enhanced ad features and an improved experience with AdWords, AdSense and other Google products by sharing your website's Google Analytics data with other Google services. *Only Google services (no third parties) will be able to access your data.* [Show example](#)
- ☒ **Anonymously with Google and others** optional
Enable benchmarking by sharing your website data in an anonymous form. Google will remove all identifiable information about your website, combine the data with hundreds of other anonymous sites in comparable industries and report aggregate trends in the benchmarking service. [Show example](#)
- ☒ **Account specialists** optional
Give Google marketing specialists and my Google sales specialists access to my Google Analytics data and account so they can find ways to improve my implementation and analysis, and share optimization tips with me. If I don't have dedicated sales specialists, give this access to authorized Google representatives.

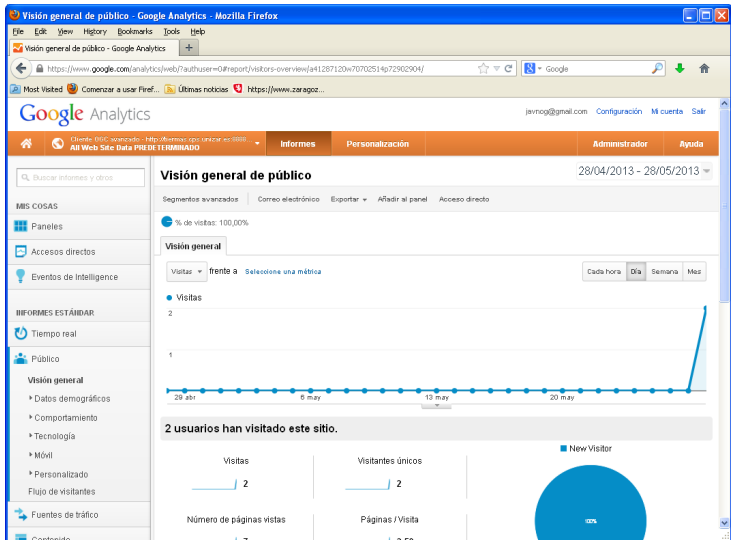
Get Tracking ID Cancel

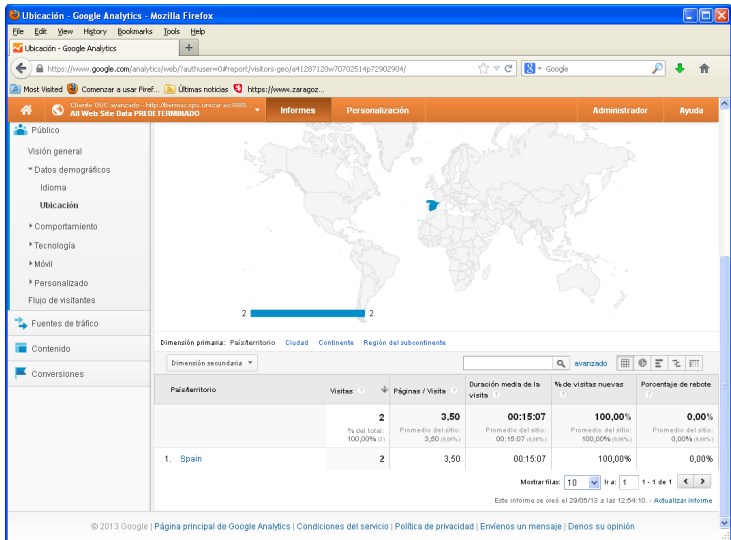
© 2013 Google | [Analytics Home](#) | [Terms of Service](#) | [Privacy Policy](#) | [Contact us](#) | [Send Feedback](#)

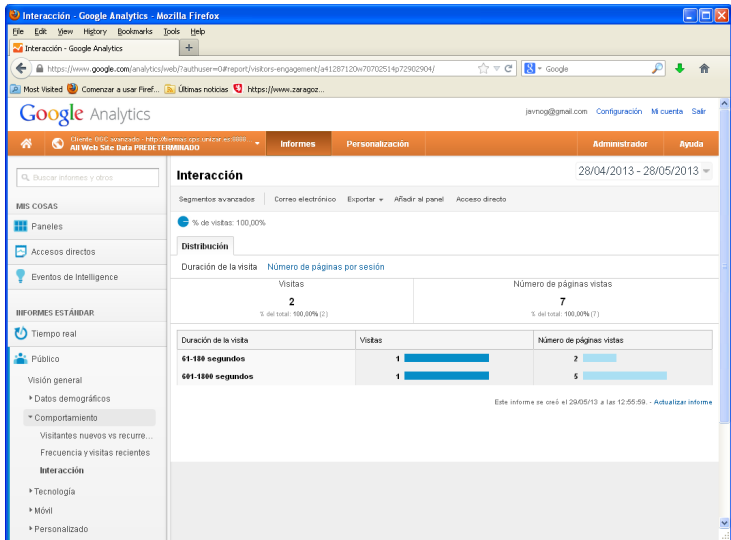
- Script que debo insertar en cada página que quiera analizar



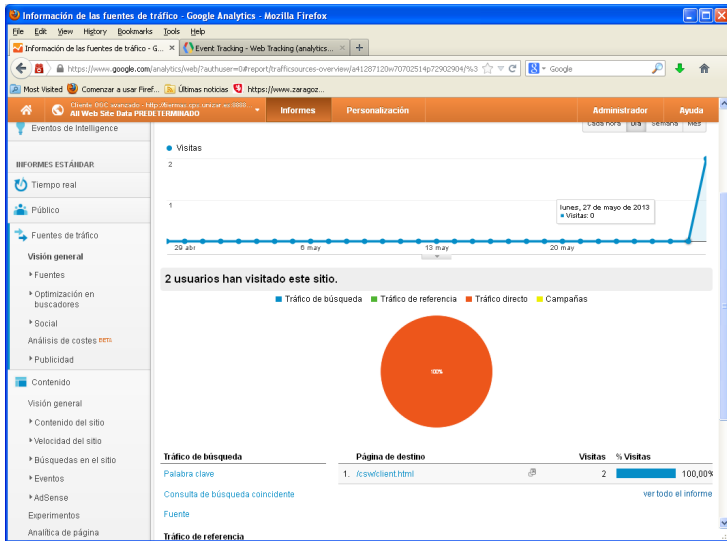
```
46      <!-- The body can have arbitrary html, or      -->
47      <!-- you can leave the body empty if you want -->
48      <!-- to create a completely dynamic ui      -->
49      <!--      -->
50      <body>
51    <script>
52      (function(i,s,o,g,r,a,m){i['GoogleAnalyticsObject']=r;i[r]=i[r]||function(){
53      (i[r].q=i[r].q||[]).push(arguments)},i[r].l=1*new Date();a=s.createElement(o),
54      m=s.getElementsByTagName(o)[0];a.async=1;a.src=g;m.parentNode.insertBefore(a,m)
55      })(window,document,'script','//www.google-analytics.com/analytics.js','ga');
56
57      ga('create', 'UA-41287120-1', 'unizar.es');
58      ga('send', 'pageview');
59
60    </script>
61
62      <!--      -->
63      <!-- This script is required bootstrap stuff. -->
64      <!-- You can put it in the HEAD, but startup -->
65      <!-- is slightly faster if you include it here. -->
66      <!--      -->
67      <!--script src="http://maps.google.com/maps?file=api&v=2&key=ABQIAAAAsSydtFb8aOVbq_zACT
68      -->
69
70      <script src="OpenLayers.js" type="text/javascript"></script>
71
72      <!-- OPTIONAL: include this if you want history support -->
73      <iframe src="javascript:''" id="__gwt_historyFrame" tabIndex='-1' style="
74
75      <!-- RECOMMENDED if your web app will not function without JavaScript enabled -->
76      <noscript>
```



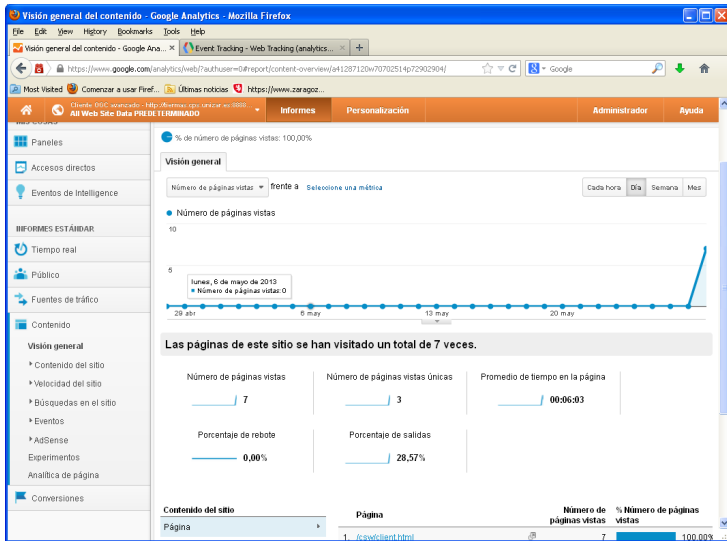




- Tráfico de búsqueda (desde buscadores)
- Tráfico de referencia (desde otros sitios web)
- Campañas (desde enlaces especiales en redes sociales)

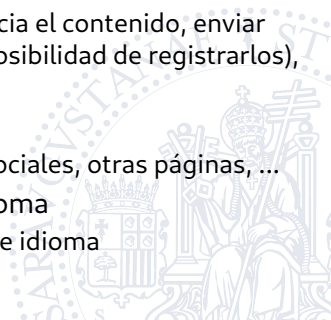


- Porcentaje de rebote: sesiones de una sola página sin interacción con el usuario



Diagnósticos y posibles soluciones

- Los usuarios dedican poco tiempo a interaccionar con nuestra Web
 - No es lo que buscan, o no encuentran una motivación para quedarse -> Convendría mejorar página principal con enlaces directos a contenidos
- Hay pocas repeticiones de visitas
 - Convendría actualizar con mayor frecuencia el contenido, enviar correos a potenciales usuarios (o dar la posibilidad de registrarlos), habilitar RSS (notificar cambios), ...
- Hay poco tráfico de referencia
 - Convendría añadir enlaces desde redes sociales, otras páginas, ...
- Hay muchas visitas de un país con otro idioma
 - Convendría tener páginas traducidas a ese idioma



Índice

1. Introducción
2. Automatización de pruebas con GUI
3. Automatización de pruebas de aceptación de usuarios
4. Soporte para pruebas de usabilidad
5. Soporte para pruebas de accesibilidad



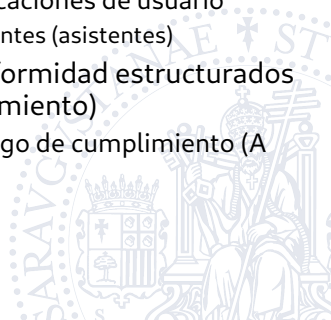
5. Soporte para pruebas de accesibilidad

- En la asignatura de IPO visteis principios de diseño universal, legislación aplicable y algunas recomendaciones respecto a la comprobación de la accesibilidad
- Verificar cumplimiento de normativa y recomendaciones (heurísticas)
 - UNE 139802: Requisitos de accesibilidad del software
 - UNE 139803: Aplicaciones informáticas para personas con discapacidad. Requisitos de accesibilidad para contenidos en la Web
 - Recomendaciones de la Iniciativa de Accesibilidad en la Web (*Web Accessibility Initiative - WAI*) de W3C
 - Han redactado Pautas de Accesibilidad para el Contenido en Web (*Web Content Accessibility Guidelines - WCAG*)
 - La UNE 139803:2012 es una trasposición de las "Pautas de Accesibilidad para el Contenido en la Web" (WCAG 2.0)
 - Acceso a normas UNE a través de los servicios electrónicos de la Universidad de Zaragoza: <https://cuarzo.unizar.es:9443/login?url=https://plataforma.aenormas.aenor.com>

- Las recomendaciones se estructuran en 4 principios y 12 pautas:
 - 1. Perceptible: contenido percibido por todos los usuarios (de forma visual, sonora, táctil, etc.)
 - 1.1 Proporcionar alternativas textuales para contenido no textual
 - 1.2 Proporcionar alternativas para contenidos dependientes del tiempo (audio, vídeo)
 - 1.3 Crear contenido que se pueda presentar de distintas formas (layout simple) sin perder información
 - 1.4 Facilitar a los usuarios que vean y oigan el contenido
 - 2. Operable: contenido manejable usando dispositivos de entrada (ratón, teclado, etc.)
 - 2.1 Acceder a toda la funcionalidad desde el teclado
 - 2.2 Proporcionar el tiempo suficiente para leer y usar el contenido
 - 2.3 Diseñar el contenido evitando estridencias
 - 2.4 Proporcionar ayudas para navegar, localizar contenido, determinar la ubicación

● ...

- 3. Comprensible: usuarios capaces de entender el contenido, su organización y su manejo
 - 3.1 Hacer el contenido textual legible y comprensible
 - 3.2 Crear páginas con apariencia y operatividad predecibles
 - 3.3 Proporcionar ayuda para evitar y corregir errores
- 4. Robusto: contenido correctamente estructurado para garantizar un adecuado funcionamiento con las aplicaciones de usuario
 - 4.1 Maximizar la compatibilidad con agentes (asistentes)
- Dentro de cada pauta hay criterios de conformidad estructurados en los niveles A, AA, y AAA (mayor cumplimiento)
 - Permiten clasificar un sitio web en un rango de cumplimiento (A menor, AAA mayor)



Sistemas automatizados de comprobación

- Herramientas de evaluación de la iniciativa WAI
 - <https://www.w3.org/WAI/ER/tools/>
- Algunas de ellas:
 - TAW (Test de Accesibilidad Web)
 - <https://www.tawdis.net/>
 - Desarrollada por la Fundación CTIC (España), Centro Tecnológico especializado en tecnologías de Internet
 - A-Checker (Accessibility Checker)
 - <https://achecker.achecks.ca/checker/index.php>
 - Desarrollada por la Universidad de Toronto (Canadá)
 - WAVE (Web Accessibility Evaluation Tool)
 - <https://wave.webaim.org/>
 - Desarrollada por la Universidad del Estado de Utah (Estados Unidos)

Ejemplo de utilización de TAW On-line

The image displays two screenshots of the Taw web accessibility testing tool interface.

Top Screenshot: Web accessibility test

The interface shows the Taw logo and a sidebar with links: Index, Services, Tools, and Contact. The main content area is titled "Web accessibility test" and features a text input field containing the URL "https://www.zaragoza.es/sede/". Below the input field is an "Analyze" button and an "Options" button. Under the "Options" button, there is a section for "Analysis level" with radio buttons for "Level A", "Level AA", and "Level AAA". Below this, it says "Level AA - Technologies: HTML, CSS, JS".

Bottom Screenshot: Summary

The interface shows the Taw logo and the same sidebar. The main content area is titled "Summary" and displays three columns of results:

- 5 Problems** in 3 success criteria. Corrections are needed.
 - Perceivable 1
 - Operable 3
 - Understandable 1
 - Robust 0
- 66 Warnings** in 30 success criteria. A human review is necessary.
 - Perceivable 43
 - Operable 13
 - Understandable 12
 - Robust 0
- 17 Not reviewed** in 17 success criteria. Fully manual review.
 - Perceivable 4
 - Operable 8
 - Understandable 4
 - Robust 1

Below the results, it shows the "Resource: https://www.zaragoza.es/sede/" and "Date: 08/05/2024 13:10 Guidelines WCAQ 2.1 Analysis level: AA Technologies: HTML, CSS". A message states: "Access the **detailed report** to obtain more information about the problems found." Below this is an email input field with a placeholder "@ email" and a "Receive report" button.

A-Checker

https://achecks.org/checker/index.php

[Login](#) [Register](#)

ACHECKER®

AChecker Web Accessibility Checker

[Subscribe for automatic accessibility monitoring with ACHECKS!](#)

Check Accessibility By:

URL **Upload** **Markup**

Address:

[Options](#)

Accessibility Review

Accessibility Review (Guidelines: [WCAG 2.0 \(Level AA\)](#))

Known Problems(0) **Likely Problems (0)** **Potential Problems (506)** **HTML Validation** **CSS Validation**

🎉 **Congratulations! No known problems.**

← → ↻ https://wave.webaim.org/report#/https://zaragoza.es

WAVE powered by WebAIM
web accessibility evaluation tool

Address: https://zaragoza.es

Styles: OFF ON

Summary

Summary Details Reference Order Structure Contrast

2 Errors	6 Contrast Errors
37 Alerts	75 Features
31 Structural Elements	198 ARIA

[View details](#)

The following apply to the entire page:

- *Logotipo Ayuntamiento de Zaragoza*
- *aria-label="Home"*
- *aria-label="Navegación principal"*
- *aria-label="Abrir Mapa web"*
- *aria-label="ReadSpeaker webReader: Escuchar con webReader"*

Zaragoza

Sede Electronica

Escuchar

¿En que podemos ayudarte?

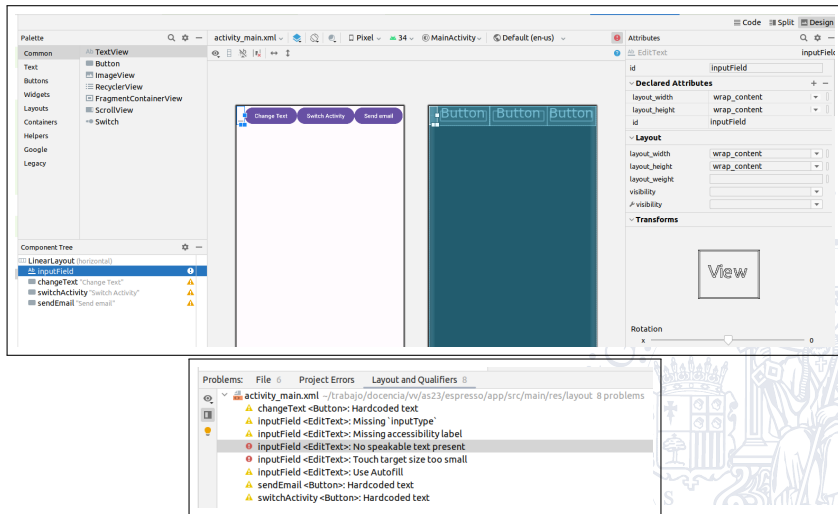
BUSCAR INFORMACION
Descubre toda la información de los temas principales de la ciudad

Evaluación de la accesibilidad en aplicaciones Android

- <https://developer.android.com/guide/topics/ui/accessibility/testing>
- Posibles aproximaciones
 - Utilización de los servicios de accesibilidad de Android. Algunos ejemplos:
 - TalkBack: aplicación que permite leer la información mostrada en la pantalla. Nos puede ayudar a encontrar problemas de falta de texto asociado a ciertos elementos gráficos, chequear si podemos acceder a todos los elementos, ...
 - Voice Access: permite controlar el dispositivo mediante comandos de voz
 - Utilización de herramientas de análisis como Accessibility Scanner para recibir sugerencias sobre etiquetas, elementos sobre los que se puede hacer clic, contraste, ...
 - Esta App integra el Accessibility Test Framework. Software que implementa el chequeo de algunas de las recomendaciones de la iniciativa WAI para móviles (<https://www.w3.org/WAI/standards-guidelines/mobile/>)

Accessibility Scanner dentro del editor de layouts

- También integrada en Android Studio dentro del editor de layouts con vista de diseño (pulsando el botón de error)



Accesibilidad Scanner dentro de pruebas automatizadas

```
import androidx.test.espresso.accessibility.AccessibilityChecks;
```

```
@RunWith(AndroidJUnit4.class)
```

```
@LargeTest
```

```
public class AccessibilityEspressoTest{
```

```
    @Rule
```

```
    public ActivityScenarioRule<MainActivity> mActivityRule = new  
        ActivityScenarioRule<>(MainActivity.class);
```

```
    @BeforeClass
```

```
    public static void enableAccessibilityChecks(){  
        AccessibilityChecks.enable();  
    }
```

```
com.google.android.apps.common.testing.accessibility.framework.integrations.espresso.AccessibilityViewCheckException: There were 2 accessibility results:  
AppCompatEditText{id=2131238943, res-name=inputField, visibility=VISIBLE, width=53, height=118, has-focus=false, has-focusable=true, has-window-focus=false,  
is-focused=false, is-focusable=true, is-layout-requested=false, is-selected=false, layout-params=android.widget.LinearLayout$LayoutParams@YYYYY, tag=null, root-is-layout-requested=false,  
has-input-connection=true, editor-info=[inputType=0x20001 imeOptions=0x40000006 privateImeOptions=null actionLabel=null actionId=0 initialSelStart=0 initialSelEnd=0 initialToolType=0 initialCapsMode=0x0  
hintText=null label=null packageName=null autoFillId=null fieldId=0 fieldName=null extras=Bundle{[android.support.text.emoji.emojiCompat_metadataVersion=0, android.support.text.emoji  
.emojiCompat_replaceAll=false]} hintLocales=null supportedHandwritingGestureTypes=SELECT|SELECT_RANGE|DELETE|DELETE_RANGE contentMimeType=null }, x=0.0, y=1.0, text=, input-type=131073, ime-target=false, has-links=false}: This item's size is 280dp x  
45dp. Consider making this touch target 32dp wide and 48dp high or larger. Reported by com.google.android.apps.common.testing.accessibility.framework.checks.TouchTargetSizeCheck,  
AppCompatEditText{id=2131238943, res-name=inputField, visibility=VISIBLE, width=53, height=118, has-focus=false, has-focusable=true, has-window-focus=false, is-clickable=true, is-enabled=true,  
is-focused=false, is-focusable=true, is-layout-requested=false, is-selected=false, layout-params=android.widget.LinearLayout$LayoutParams@YYYYY, tag=null, root-is-layout-requested=false,  
has-input-connection=true, editor-info=[inputType=0x20001 imeOptions=0x40000006 privateImeOptions=null actionLabel=null actionId=0 initialSelStart=0 initialSelEnd=0 initialToolType=0 initialCapsMode=0x0  
hintText=null label=null packageName=null autoFillId=null fieldId=0 fieldName=null extras=Bundle{[android.support.text.emoji.emojiCompat_metadataVersion=0, android.support.text.emoji  
.emojiCompat_replaceAll=false]} hintLocales=null supportedHandwritingGestureTypes=SELECT|SELECT_RANGE|DELETE|DELETE_RANGE contentMimeType=null }, x=0.0, y=1.0, text=, input-type=131073, ime-target=false, has-links=false}: This item may not have a  
label readable by screen readers. Reported by com.google.android.apps.common.testing.accessibility.framework.checks.SpeakableTextPresentCheck
```

¿Cuestiones?



Departamento de
Informática e Ingeniería
de Sistemas
Universidad Zaragoza

